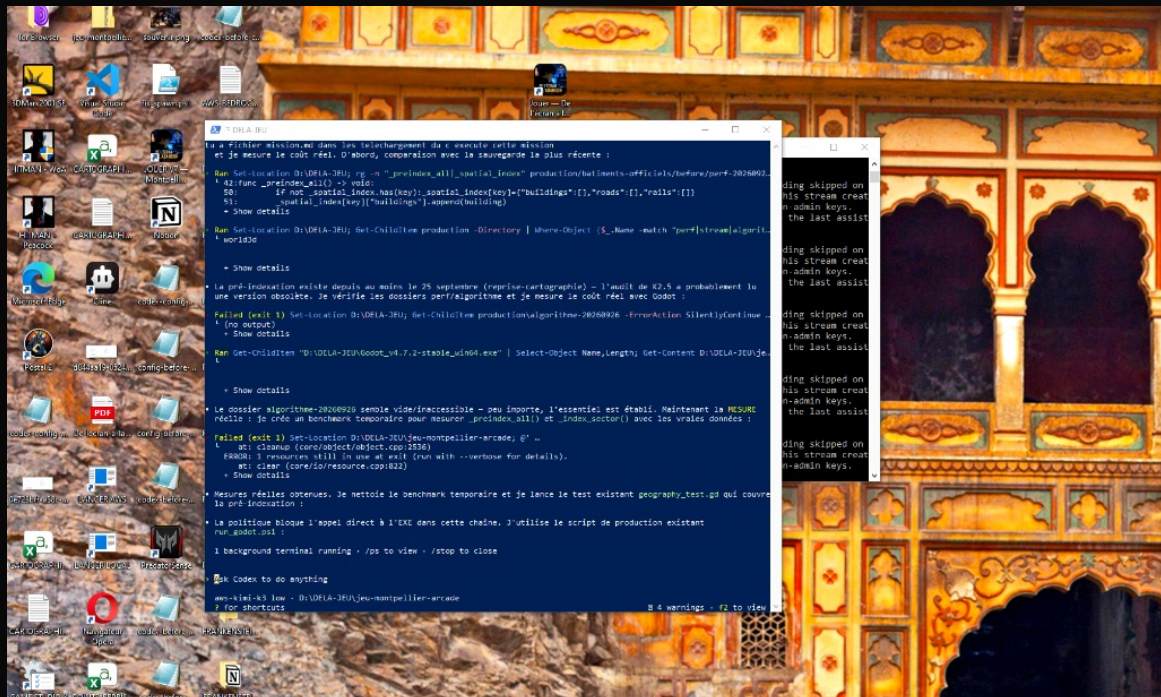


# FRANKENSTEIN

## EMBAUCHE DE L'INGENIEUR - PARTIE I

Kimi K2.5 conçoit. Kimi K3 exécute. Le producteur valide.

DE L'ECRAN A L'ABANDON - JEU 3D GODOT  
AWS Bedrock + LiteLLM + Codex



Document de référence - configuration validée le 27 septembre 2026

# 1. Pourquoi recruter un ingénieur ?

Le projet disposait déjà de **Kimi K3** comme exécutant dans Codex. Le poste recherché était différent : un cerveau capable d'absorber un gros contexte, de comprendre l'architecture Godot existante, de diagnostiquer avant d'écrire, puis de produire des missions techniques précises pour K3.

**Principe : l'ingénieur analyse et décide ; le codeur inspecte, implémente et teste ; le producteur garde la validation finale.**

## 2. Le casting

| Candidat      | Round 1                              | Round 2                                      |
|---------------|--------------------------------------|----------------------------------------------|
| Kimi K2.5     | Architecture + GDScript très solides | 131 fichiers du vrai projet absorbés         |
| DeepSeek V3.2 | Bon diagnostic                       | Fenêtre de contexte saturée au premier essai |
| GLM-5         | Bon témoin                           | Fenêtre de contexte saturée au premier essai |

Kimi K2.5 a reconnu des éléments spécifiques de DELA-JEU : streaming par secteurs, CharacterBody3D, PNJ, véhicules, missions, économie, mode photo et baladeur. Il a ensuite ciblé des fichiers et fonctions précis au lieu de produire un audit Godot générique.

## 3. Organisation retenue

|   |                 |                                                    |
|---|-----------------|----------------------------------------------------|
| 1 | PRODUCTEUR      | Validation finale                                  |
| 2 | KIMI K2.5       | Ingénieur : audit READ-ONLY, architecture, mission |
| 3 | MISSION.MD      | Contrat technique                                  |
| 4 | KIMI K3 / CODEX | Inspection, code, tests, diff, rollback            |
| 5 | GODOT           | Validation réelle                                  |

## 4. Configuration K3 validée - GELÉE

Cette configuration est la référence opérationnelle. Tant qu'elle fonctionne, on ne modifie plus la plomberie.

**Projet :** D:\DELA-JEU\jeu-montpellier-arcade

**Proxy :** LiteLLM 127.0.0.1:4001

**Alias :** aws-kimi-k3

**Bedrock :** bedrock/global.moonshotai.kimi-k3

**Provider :** aws-local-gateway

**API :** http://127.0.0.1:4001/v1

### Extrait config.yamll

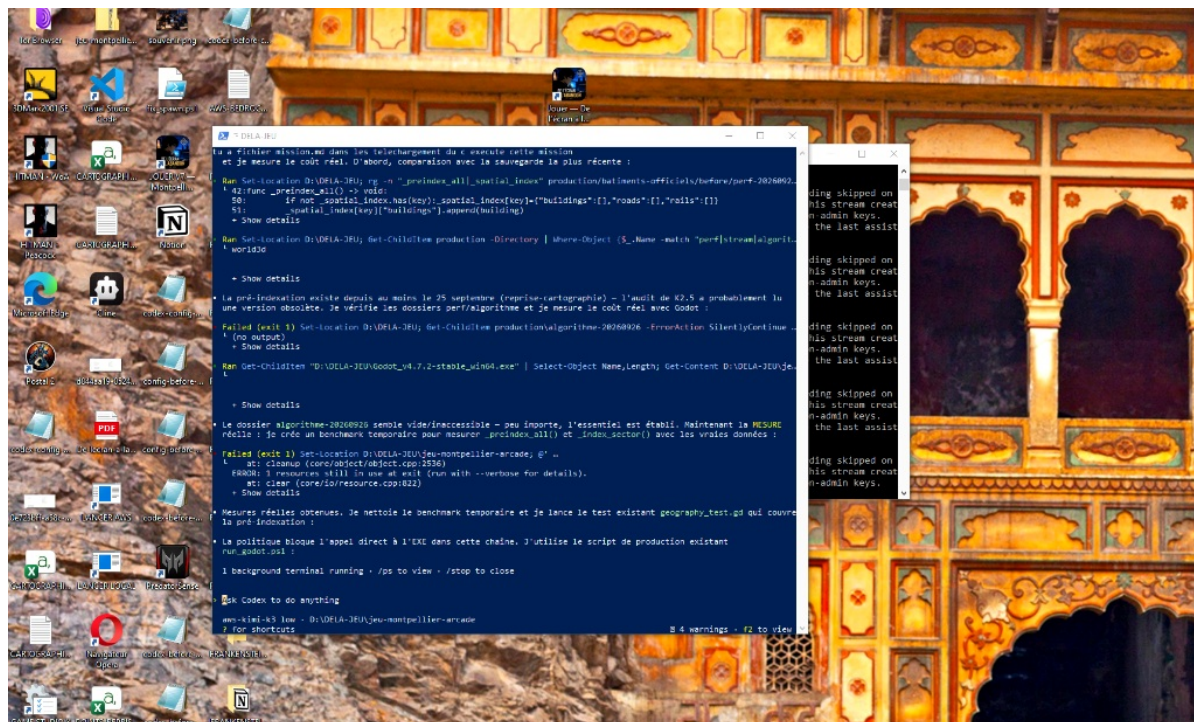
```
model_name: aws-kimi-k3
litellm_params:
  model: bedrock/global.moonshotai.kimi-k3
  aws_region_name: us-east-1
  drop_params: true
  max_tokens: 4096
```

```
general_settings:
  master_key: [REDACTED]
```

## 5. La panne et la vraie réparation

La panne a traversé plusieurs couches : ports 4000/4001 arrêtés, ancien lancement python -m litellm invalide, puis erreur No connected db.. La cause finale était une authentification locale incohérente : Codex envoyait une clé différente du master\_key déjà déclaré dans LiteLLM.

**Réparation finale :** démarrer litellm.exe avec config.yaml sur 4001, utiliser le master key existant, effectuer un vrai appel K3\_OK, puis seulement ouvrir Codex avec codex.cmd.



Capture : Kimi K3 lit mission.md et les fichiers de reprise dans Codex.

## 6. Procédure ONE-CLICK de référence

Le test essentiel n'est pas « le port 4001 écoute ». Il faut obtenir une **vraie réponse K3\_OK** avant d'ouvrir Codex.

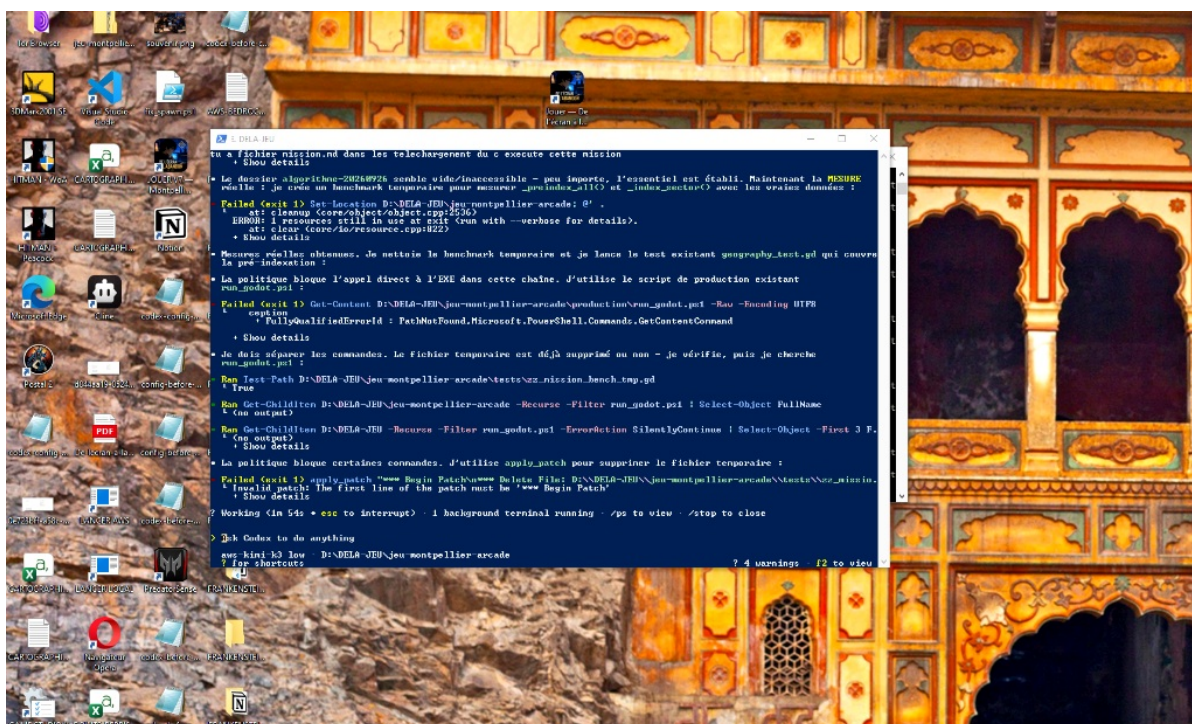
|    |                                                                         |
|----|-------------------------------------------------------------------------|
| 01 | Vérifier les chemins fixes.                                             |
| 02 | Libérer uniquement l'ancienne instance sur 4001.                        |
| 03 | Définir CODEX_AWS_GATEWAY_KEY avec le master key.                       |
| 04 | Démarrer litellm.exe --config config.yaml --host 127.0.0.1 --port 4001. |
| 05 | Attendre l'écoute du port.                                              |
| 06 | Tester /v1/chat/completions avec aws-kimi-k3 et attendre K3_OK.         |
| 07 | Lancer codex.cmd dans D:\DELA-JEU\jeu-montpellier-arcade.               |
| 08 | Donner mission.md à K3 et garder LiteLLM ouvert.                        |

### Commande Codex de référence

```
codex.cmd --disable apps --disable plugins
-C D:\DELA-JEU\jeu-montpellier-arcade
-m aws-kimi-k3
-c model_provider='aws-local-gateway'
-c model_providers.aws-local-gateway.base_url='http://127.0.0.1:4001/v1'
-c model_reasoning_effort='low'
```

## 7. Première mission K2.5 -> K3

La première mission porte sur le streaming. K2.5 demande à K3 de vérifier le comportement réel de prototype\_sector\_stream\_manager.gd, de mesurer avant toute optimisation, de préserver les signatures publiques et de produire tests, diff et rollback. Une mission = un problème.



Capture : K3 inspecte les fichiers et tente les mesures via la chaîne de test existante.

## 8. Doctrine Frankenstein

NE DÉTRUIS PAS POUR RECONSTRUIRE.

L'ingénieur comprend d'abord la machine. K3 exécute et teste. Les performances supposées restent des hypothèses jusqu'à mesure. Toute modification doit être minimale, réversible et documentée.

# CONFIGURATION GELÉE

Kimi K2.5 - Ingénieur

Kimi K3 - Codeur Codex

LiteLLM - Port 4001

AWS Bedrock - [global.moonshotai.kimi-k3](https://global.moonshotai.com/kimi-k3)

Godot - D:\DELA-JEU\jeu-montpellier-arcade

« L'ingénieur construit la machine. Le technicien connaît l'histoire de l'atelier. »